Experiment 3: Acceleration during Pendulum Motion

ME-330

Dr. Vibhav Durgesh

vdurgesh@uidaho.edu

Week-03

Contents

1 Learning objective 3

2 Equipment checklist 3

3 Introduction 3

4 Laboratory instruction 3

5 Part-A: Matlab code 4

6 Part-B: Circuit 4

7 Part-C: Test circuit 6

8 Part-D: Calibration 6

9 Part-E: Modifying acquisition code 6

10 Part-F: Normal acceleration experiment 7

11 Part-G: Plotting data **7**

12 Part-H: Clean up **8**

13 Instruction **8**

14 Post lab (50 points) **9**

15 Appendix **10**
15.1 Matlab program to acquire data for calibration 10
15.2 Matlab program to generate calibration plot 11
15.3 Matlab program to acquire data 12
15.4 Matlab program to plot acceleration data 14

1 Learning objective

The objectives of this experiment are to provide the student with an opportunity to:

- Gain additional experience with Matlab.
- Create a simple program for data acquisition using toolbox and National Instrument's NI-MYDAQ.
- Become familiar with breadboards, digital power supply, wiring, digital multi-meters, and other laboratory skills.
- Perform calibration of a MEMS accelerometer and create the calibration plot.
- Compare measured normal acceleration to acceleration calculated from angular velocity during fixed axis rotation.

2 Equipment checklist

- USB Data Acquisition Device (NI MyDAQ).
- Pendulum with Potentiometer ($10k\Omega$ Bourns 6639s) pivot and end-mounted accelerometer (MMA7361LC).
- Needle nose pliers & wire strippers.
- Breadboard – Digilent
- Small flathead screwdriver.
- Digital multi-meter (DMM)
- Power supply and wire kit.

3 Introduction

During rotation about a fixed axis, the normal acceleration of a point can be calculated from the angular velocity of the rotation motion according to the following formula:

$$a_n = -\dot{\theta}^2 r \tag{1}$$

4 Laboratory instruction

This experiment will build on your work from the previous week. In addition to measuring the pendulum angle with a calibrated potentiometer, you will also measure normal acceleration of the end of the pendulum using a MEMS accelerometer. Schematic is shown in figure 1. Start by setting up the NI MyDAQ to acquire data from the potentiometer and accelerometer. Next, you will calibrate the y-axis of the accelerometer. Finally, we will compare measured normal acceleration of the pendulum end to normal acceleration calculated from the derivative of the pendulum angle.

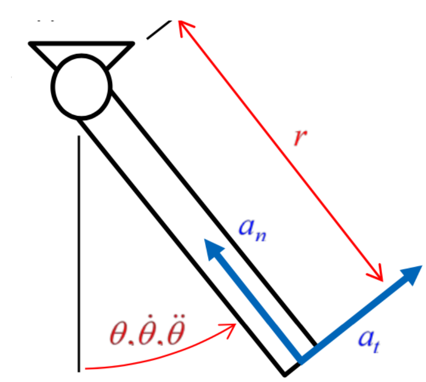


Figure 1: Schematic of the experimental setup.

5 Part-A: Matlab code

Creating data acquisition program using Matlab data acquisition toolbox
A sample Matlab program is provided in the appendix section of the instruction use that program as the starting point and modify accordingly. The modified Matlab data acquisition program should acquire data from two-channels of the NI-MyDAQ, display and record the data.

- Make sure that matlab program is acquiring data from two channels, The first channel should be acquiring potentiometer angle data and the second channel is acquiring the MEMS accelerometer data
  - Set the sample rate to 1000 samples/sec. We will be calculating a discrete derivative of the pendulum angle and a higher sample rate will reduce the noise from the process.
- The plot should be displaying the voltages from the accelerometer y-axis. You will need these for calibration of accelerometer.
- Please use the calibration equation from the previous laboratory experiment. If in doubt please contact the TAs for the correct calibration. The calibration should be able to convert the potentiometer output voltage to radians. Program this equation into the Matlab acquisition program. Verify that the calibration is producing the correct output by moving the pendulum to several known angles.
- Measure the length of the pendulum are from Pivot point to accelerometer location. This will be used to calculate acceleration from angular position data.

6 Part-B: Circuit

Connect the y-axis accelerometer output to the data acquisition using a breadboard.

- For this experiment, a 3-axis accelerometer has been fixed to the end of the pendulum apparatus (see Figure 2). The accelerometer has been configured for ±6g operation. The y-axis of the pendulum points towards the pendulum rotation axis (see Figure 3). For more information about the accelerometer, see Pololu.
- Connect the accelerometer to the breadboard using the provided 3-wire cable. See figure 3(a).
- Connect the Vin and GND pins on the accelerometer to the 5 Volt supply power (already used to power the potentiometer). See figure 3(b) for reference.
- Connect the y-axis accelerometer output to AI1 (Differential Analog Input 1). See figure 3(b).
- Connect the DAQ GND, instead of the AI1-, to the ground on the breadboard. See figure 3(b).

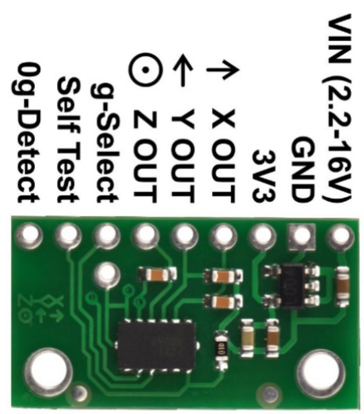
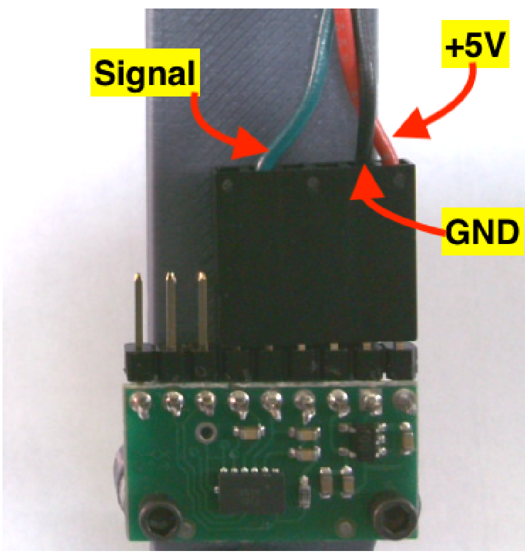
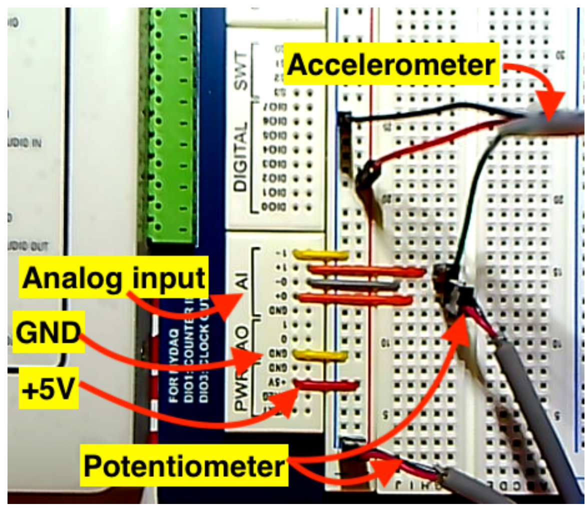


Figure 2: The MMA7361LC 3-axis accelerometer with voltage regulator.



(a) Sample connection to the accelerometer.



(b) Sample connection to the DAQ system using Digilent board .

7 Part-C: Test circuit

Test your acquisition program and test data acquisition system

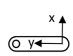
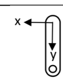
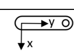
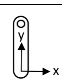
- Make sure that you are acquiring two channels of data from DAQ system
- When you have verified your circuit and connections, run your Matlab program.
- Modify the calibration bias as necessary to zero the pendulum as is hangs at the bottom.
- Confirm that you are acquiring data from the accelerometer. You will need the y-axis accelerometer voltages for the calibration step that follows (Part-D).

8 Part-D: Calibration

Calibrate the y-axis accelerometer outputs.

- Complete the table below in your logbook by rotating the pendulum to the four shown positions and recording the output voltages on the y-axis of the accelerometer. Collect the voltage output from accelerometer for 20 seconds at a sampling rate of 20Hz. Make sure that you have saved these calibration file, if you need to verify your calibration. Fill the information in the table provided below. Please make sure that you are using correct channel in your data acquisition system.
- Plot the data points from the above table in order to create calibration equations for the y-axis acceleration measurements. Specifically, Plot ay_g vs. $ay_{v,avg}$ to find y-axis acceleration (in m/s^2) as a function of sensor voltage.

Table 1: Sample calibration data for experiment 2.

Pendulum orientation	Schematic	y-axis acceleration ay_g (m/s^2)	Average output voltage $ay_{v,avg}$ (v)
Right		0.00	
Up		9.81	
Left		0.00	
Down		-9.81	

9 Part-E: Modifying acquisition code

Now add your calibration equation to acquisition program to display the pendulum angle in degrees during the data acquisition process and acceleration values in m/s^2.

- You will have to change the slope and intercept values in the provided Matlab program.
- Run your Matlab program for few seconds and see if your program displays the correct angle and acceleration values.
- Show your program and results to the TAs to verify your program. **If program is working properly, the proceed to collect data.**

10 Part-F: Normal acceleration experiment

Procedure for collecting data

- Change the sampling rate to 500 hz and sampling time to 15 seconds. Use the code provide in section 15.3.
- Make sure the output file is named as "Expt_03_FirstName_Lastname.dat"
- Make sure to input the appropriate slope and intercept values to convert voltage data from potentiometer to angles in degrees and voltage from accelerometer to acceleration.
- Move the pendulum the horizontal position location.
- Start the acquisition program
- Release the pendulum
- Make sure the plot shows pendulum angular location and pendulum normal acceleration.
- Show the final output figure to the TAs.
- Open your output file and check the data.
- Output file should have three columns of data. The first column will have time information, second column will have angular position of pendulum in degrees, and the third column will have acceleration value from accelerometer in m/s^2.

11 Part-G: Plotting data

Post processing of the data

- Open the file using Matlab program and read all three columns of data.
- Convert the angle value in radians
- Remove the acceleration due to gravity from the accelerometer data
  - Here you have to use free-body diagram to identify the contribution from acceleration due to gravity
  - Use the angular value for pendulum to accurately identify the position of the pendulum.
  - The result will be the measured normal acceleration of the pendulum due to angular velocity.
- Plot the normal acceleration data from accelerometer and normal acceleration using equation 1 in a single plot.
- Plotting the noisiest data first (This way the other data will remain visible). Sample plot is shown in Figure 3.
- Code to create post-process file is shown in appendix 15.4

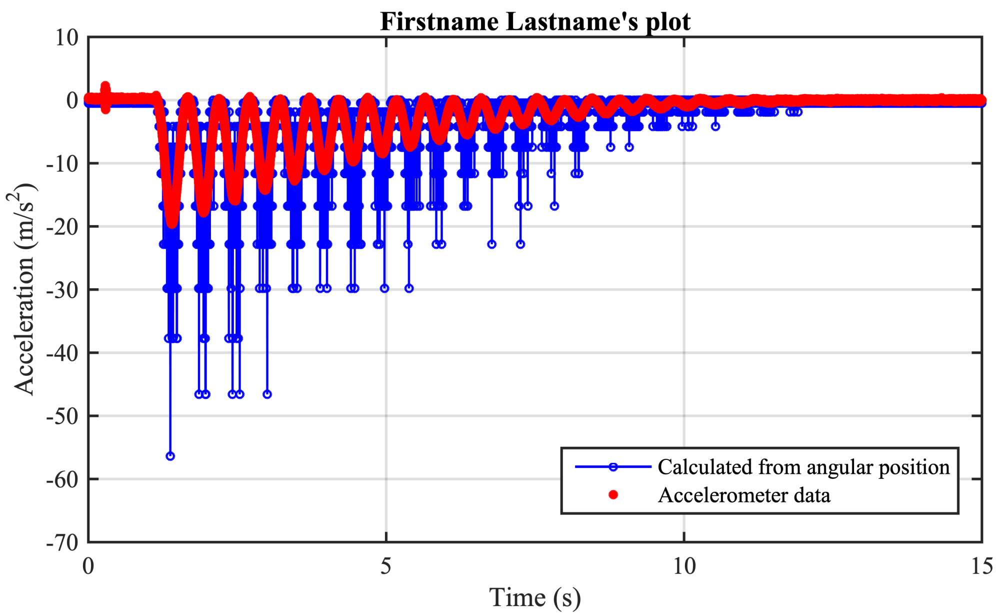


Figure 3: Sample post-process plot.

12 Part-H: Clean up

Return lab space to prior condition

- Turn off the power supply.
- Remove wires connecting the power supply to the breadboard and return to the wall
- Remove all breadboard wires and place them back in the wire kit in an organized fashion.
- Remove the pendulum wires from the breadboard and set aside.
- Log off the computer.
- Make sure to copy the data to your harddrive.
- **Do this for every lab!**

13 Instruction

- Make sure to name the files your FirstName_ LastName_ QuestionNumber, use the format provided in the sample Matlab code. Make sure the submitted figure has the title. Include the apostrophe!
- Make sure to submit your post-lab assignment on the Canvas website.
- Here are some details to help.
  - Plot the experimental data using red circles with MarkerSize 8. Experimental data points should always be plotted using markers without lines connecting the points
  - Plot curve from theory (or curve fit line) using a solid colored line with LineWidth 2.
  - Set the x and y-axis limits for each figures. Follow the instruction.
  - Make sure the major grid lines are visible

- All plot text should be in Times font. You will need to specify this for the title, labels, and legends. The axis labels and numbers should be in 10-point font
- The title should be in 10-point font
- Make sure to add the legend. Keep in mind that legend should not hide the plotted data
- For full credit make sure that your submitted files look similar to the sample figures shown in the document
- All the submitted plot should be in png format with 600 dpi resolution.
- If in doubt make sure to request the TAs for help

14 Post lab (50 points)

Postlab are due at Friday 5:00 pm. For this experiment submit the following items on Canvas for your post-laboratory assessment.

1. Accelerometer Calibration Plot. Submit a calibration plot showing the raw data points as markers and the calibration curve as a line. The plot should be 6.5inch wide. Make sure to properly annotate your plots: axis labels, titles, legend, etc. Using the text command in Matlab, place the equations or numbers listed below on the plot. Use Greek symbols where appropriate.
   (a) The calibration equation on the plot with 4 significant digits for each number.
   (b) The norm of the residuals of your linear regression.
   (c) The standard error of the fit for your calibration equation (Make sure to include appropriate units.)
2. Pendulum Swing Plot. Submit a time series plot showing normal acceleration vs. time calculated in the two ways described in Part-G.
   (a) Normal acceleration measured from the accelerometer vs. time.
   (b) Normal acceleration calculated from the equation $a_n = \dot{\theta}^2 r$.
   (c) The plot should be 6.5inch wide. Make sure to properly annotate your plots: axis labels, titles, legend, etc. Use Greek symbols where appropriate.
3. Answer all question in the post-lab assessment on Canvas.

15 Appendix

15.1 Matlab program to acquire data for calibration

```
% This is a generic program to aquire the data from myDAQ
% Sample data calibration program is for accelerometer
% it logs the data in a file for later use
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Date: August 5th, 2020
% Dr. Vibhav Durgesh
% Rev 0.0
% User has to provide appropriate information - see beginning of code
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% THIS CODE OVERWRITES THE DATA. PLEASE MOVE THE FILES OR RENAME PRIOR TO
% RERUNNING THE CODE
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
clear all
close all
clc

%% Start of user information
Fs = 20; %Sampling rate data per sec
T = 20; %Time to aquire data
filename = '../Data/Cal_Data_0.dat'; %To store calibration data file
fid1 = fopen(filename,'w');
%% End of user required information ensure channel number is correctly set for accelerometer
d = daq.getDevices;
s = daq.createSession('ni');
addAnalogInputChannel(s,'myDAQ1',['ai1'],'Voltage'); % Make sure to use the right channel for accelerometer data
s.Rate = Fs; %Sampling rate modify as required
s.DurationInSeconds = T; %Sampling time modify as required
lh1 = addlistener(s,'DataAvailable',@(src, event)plotAndLogData(src, event));
lh2 = addlistener(s,'DataAvailable',@(src, event)logData(src, event,fid1));
errorListener = addlistener(s, 'ErrorOccurred',@(src, event) disp(getReport(event.Error)));
drawnow
s.IsContinuous = true;
startBackground(s);
pause (T)
delete (s)
delete(lh1)
delete(lh2)
fclose(fid1)
data = readmatrix(filename);
avgVolt = mean(data(:,2));
gcf;
title(['Average voltage: ' sprintf('%3.4f',avgVolt) ' (v)'],'fontname','times','fontsize',14)

function plotAndLogData(src,event)
time = event.TimeStamps;
voltage = event.Data; %Accelerometer data in voltage
figure(1)
plot(time, voltage,'k.-');hold on
xlabel('time (s)')
ylabel('Voltage (v)')
end

function logData(src, evt, fid)
%Add the time stamp and the data values to data. To write data sequentially
%transpose the matrix
data = [evt.TimeStamps evt.Data];
fprintf(fid,'%f \t %f \n', data');
end
```

15.2 Matlab program to generate calibration plot

```matlab
% This is a generic program to generate calibration plot and provide slop
% and intercept data
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Date: August 5th, 2020
% Dr. Vibhav Durgesh
% Rev 0.0
% User has to provide appropriate information - see beginning of code
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
clear all
close all
clc

%% User have to manually add information
x = [1.698 1.4901 1.6923 1.9031]; %Voltage Data
y = [0 9.81 0 -9.81]; % Acceleration
t_nuP= 4.303; % Value read from t-student’s table
plotTitle = ’FirstName LastName’’s Calibration Plot’;
%% Performing linear curve fitting
[p,s] = polyfit (x,y,1); % Curve fitting with 1st order fit
%% Generating curvefit line
xfit = x;% Generating degree of freedom for the curve fit
yfit = polyval(p,xfit);
nu = s.df; % Getting degree of freedom for the curve fit
norm = s.normr; % Getting the norm of the curve fitting
syx = norm/sqrt(nu);

%% Generating figure with specific size
figure(1)
set(gcf,’unit’,’inches’,’position’,[0.50 0.50 6.50 3.50],...
    ’defaultaxesfontsize’,10,’defaultaxesfontname’,’times’);
%% Plotting data and adding labels
plot(xfit,yfit,’b-’,’linewidth’,2);hold on
plot(x,y,’ro’,’markersize’,9,’markerfacecolor’,’r’)
xlabel(’Volts (v)’)
ylabel(’Acceleration (m/s^{2})’)
%% Now adding 95% confidence level in the plot
y_cl_low = yfit - t_nuP*syx; % Using regression analysis
y_cl_up  = yfit + t_nuP*syx;
plot(xfit,y_cl_low,’--’,’color’,[0.2 0.2 0.2], ’HandleVisibility’,’off’);
plot(xfit,y_cl_up,’--’,’color’,[0.2 0.2 0.2]);

%% Adding confidence level lines on curve fit
legend(’Curve fit’,’Expt. data’,’95% Cl range’,’location’,’Northeast’)
title(plotTitle)
text(1.5,-5,sprintf(’y = %3.4fx + %3.4f’,p(1),p(2)),’Fontname’,’times’)
text(1.5,-7,sprintf(’Norm = %3.4fx’,s.normr),’Fontname’,’times’)
text(1.5,-9,sprintf(’s_{yx} = %3.4f’,syx),’Fontname’,’times’)
%% Saving the files in png and pdf format
figName = [’../Figures/Student_Name_Exp03_Part1’];
set(gcf,’PaperPositionMode’,’auto’)
print(figName,’-dpng’,’-r600’)
set(gcf,’PaperUnits’,’inches’,’Units’,’inches’);
figpos = get(gcf,’Position’);
set(gcf,’Papersize’,figpos(3:4),’Units’,’inches’);
print(figName,’-dpdf’,’-r600’)
```

15.3 Matlab program to acquire data

```
% This is a generic program to aquire the data from myDAQ
% Sample data acquisition program for calibration purpose it logs the data
% in a file for later use
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Date: August 5th, 2020
% Dr. Vibhav Durgesh
% Rev 0.0
% User has to provide appropriate information - see beginning of code
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% THIS CODE OVERWRITES THE DATA. PLEASE MOVE THE FILES OR RENAME PRIOR TO
% RERUNNING THE CODE
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
clear all
close all
clc

%% Start of user information
Fs = 1000; %Sampling rate data per sec
T = 15; %Time to aquire data
filename = '../Data/Student_Name_Expt_03.dat'; %To file to store data
fid1 = fopen(filename,'w');

slope1 = -73.915; %Add your calibration slope for potentiometer
intercept1 = 190.4438; %Add you calibration intercept for potentioment

slope2 = -47.4958; %Add your calibration slope for accelerometer
intercept2 = 80.5470; %Add you calibration intercept for accelerometer

plotTitle = 'Student''s Name Plot - Experiment#3';
fprintf(fid1,'%s \t %s \t %s \n','times (s)','Angular position (degrees)','Accerleration (m/s^{2})'); % Header for the output file
%% End of user information
d = daq.getDevices;
s = daq.createSession('ni');
addAnalogInputChannel(s,'myDAQ1',[0 1],'Voltage'); % Channel-1 is pot and #2 is accelerometer
s.Rate = Fs; %Sample rate modaify as required
s.DurationInSeconds = T; %Sampling time modify as required
lh1 = addlistener(s,'DataAvailable',@(src, event)plotAndLogData(src, event,slope1,intercept1,slope2,intercept2));
lh2 = addlistener(s,'DataAvailable',@(src, event)logData(src, event,fid1,slope1,intercept1,slope2,intercept2));
errorListener = addlistener(s, 'ErrorOccurred',@(src, event) disp(getReport(event.Error)));
drawnow
s.IsContinuous = true;
startBackground(s);
pause(T)
delete(s)
delete(lh1)
delete(lh2)
fclose(fid1)
title(plotTitle,'fontname','times','fontsize',14)

%% Function to plot the data during acquistion
function plotAndLogData(src, event, m1, c1, m2, c2)
time = event.TimeStamps;
angularPosition =  m1*event.Data(:,1) + c1; %Potentiometer data in voltage
acc = m2*event.Data(:,2) + c2; %Accerlerameter
figure(1)
plot(time, angularPosition,'k.-');hold on
plot(time, acc,'b.-')
xlabel('time (s)')
ylabel('Angular position (degrees)/ Accerleration (m/s^2)')
end

%% Function for storing the data in physical variables - 3 columns time, angle, accelrometer
```

```
function logData(src,evt, fid,m1,c1,m2,c2)
%Add the time stamp and the data values to data. To write data sequentially
%transpose the matrix
data = [evt.TimeStamps m1*evt.Data(:,1)+c1 m2*evt.Data(:,2)+c2];
fprintf(fid,'%f \t %f \t %f \n', data');
end
```

15.4 Matlab program to plot acceleration data

```matlab
% Sample program to process your data for post-lab
%%%%% Make sure to modify as required %%%%%%
close all
clear all
clc

%% Prefined Parameters get these values during the experiment
r = 0.2159; %Length of the pendulum (m)
g = -9.81; %Constant gravity (m/s^2)
%% Start user required information Reading data from the file 3 column data
fid = fopen('../Data/Student_Name_Expt_03.dat');
linel = fgetl(fid);
data = fscanf(fid, '%f \t %f \t %f \n', [3 inf]);
%% Storing data in the right variables
t = data(1,:); % Time data
y1 = data(2,:); % Angular location (degree)
y2 = data(3,:); % Accelerometer data (m/s^2)
fclose(fid);
%% Converting degrees to radian
rad = y1*pi/180; % convert degree value to radians
%% Performing numerical differentiation -  dx/dt to get angular velocity
theta_d =  diff(rad) ./ diff(t); % calculating angular velocity rad/s
%% Calculating acceleration from two different methods
acc_1 =  - (theta_d.^2) * r; % calculated acceleration m/s^2
acc_2 = y2 - (g*cos(rad)); % removing acceleration due to gravity component accelerometer data


%% Generating figure with specific size
figure (1)
set(gcf,'unit','inches','position',[0.50 0.50, 6.50 3.50],...
    'defaultaxesfontsize',10,'defaultaxesfontname','times');
%% Plotting data
plot(t(1:end-1),acc_1,'-bo','markersize',3);hold on
plot(t,acc_2,'ro','markersize',3,'markerfacecolor','r');
xlabel('Time (s)')
ylabel('Acceleration (m/s^2)')
grid on

ylim([-70 10])
xlim([0 15])

legend('Expt. data','Accelerometer data','location','Southeast')
title('FirstName LastName''s Name Plot')
%% Saving the files in png and pdf format
figName = ('../Figures/Student_Name_Expt03_Postprocess');
set(gcf,'PaperPositionMode','auto')
print(figName,'-dpng','-r600')
set(gcf,'PaperUnits','inches','Units','inches');
figpos = get(gcf, 'Position');
set(gcf, 'PaperSize', figpos(3:4),'Units','inches');
print(figName,'-dpdf','-r600')
```